Adaptive Retrieval for Open-Domain QA over Wikipedia: A Router-Guided RAG Inspired by Self-RAG

Ander Peña
University of the Basque Country (EHU), Donostia, Basque Country
apena080@ikasle.ehu.eus

Introduction

- Large language models often hallucinate factual answers
- RAG adds external passages, but always retrieving is expensive and can even hurt when passages are noisy.
- This project studies **adaptive retrieval** for Wikipedia QA: a small router decides when to retrieve, inspired by Self-RAG.

Related Work

- Self-RAG:
  - Self-RAG (Asai et al., 2023) trains a generative model to retrieve, generate, and critique through self-reflection.
  - Special reflection tokens let the model request retrieval and critique its own answers.
  - It shows that on-demand retrieval can improve factuality while using fewer documents.
- Dense Passage Retrieval:
  - DPR (Karpukhin et al., 2020) uses dense encoders for questions and passages and achieves high recall on Natural Questions.
  - In this project a DPR-style dense retriever (MiniLM) is used and focus training on the router.

Dataset and Setup

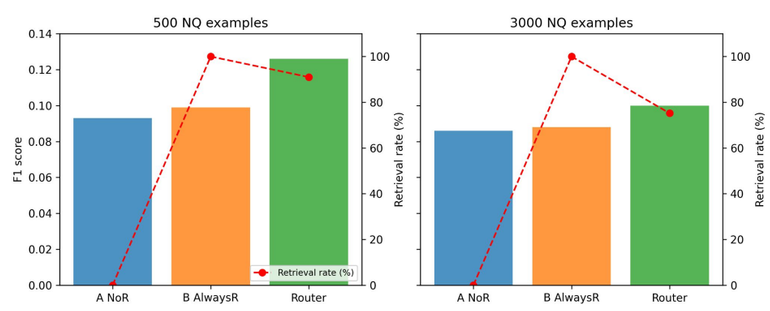
- Natural Questions (NQ): real user queries with short answers annotated on Wikipedia pages.
- I use the validation split in streaming mode, without downloading the full dataset.
- For each question, I use its associated Wikipedia page as the document, not a full Wikipedia index (document-controlled setting).
- I experiment with three sample sizes:
  - 50 questions (pipeline development)
  - 500 questions (main experiments)
  - 3000 questions (scaling analysis)

System Overview

- QA backbone
  - RoBERTa SQuAD2, extractive QA model.
  - Input: question + context
  - Output: short answer span.
- Baseline A: No-retrieve
  - Clean Wikipedia page with BeautifulSoup and feed full document to the QA model.
- Baseline B: Always-retrieve
  - Split document into 100-token chunks.
  - Use MiniLM to rank chunks.
  - Concatenate top-k chunks and feed question + chunks to the QA model.
- Router-Guided RAG (Proposed)
  - DistilBERT + logistic regression predicts P(retrieve | q).
  - If P(retrieve | q) > threshold → Baseline B, else Baseline A.

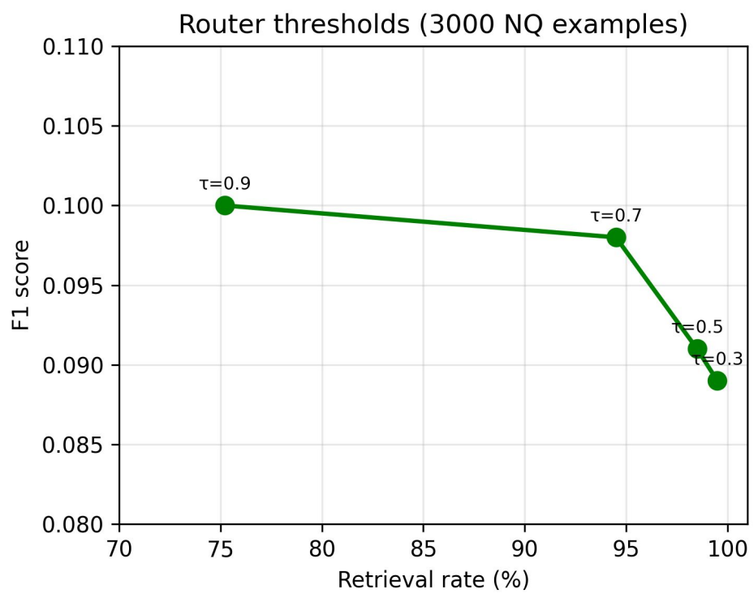
Main results table

NQ sample	System	F1	Retrieval rate
50	No-retrieve	0.090	0%
50	Always-retrieve (doc chunks)	0.068	100%
50	Router (τ = 0.5)	0.110	94%
500	No-retrieve	0.093	0%
500	Always-retrieve (doc chunks)	0.099	100%
500	Router (τ = 0.7)	0.126	91%
3000	No-retrieve	0.086	0%
3000	Always-retrieve (doc chunks)	0.088	100%
3000	Router (τ = 0.9)	0.100	75%



Router threshold vs F1 and retrieval rate

Threshold τ	F1	Retrieval rate
0.3	0.089	99.5%
0.5	0.091	98.5%
0.7	0.098	94.5%
0.9	0.100	75.2%



Increasing the threshold reduces how often the router calls the retriever. Around τ = 0.9 we get the best F1 with only 75% retrieval rate, showing a clear quality-cost trade-off.

Conclusions

- RAG performance depends not only on the retriever, but also on corpus construction and context selection; naïve Wikipedia chunking can lead to almost random retrieval.
- In a document-controlled NQ setting, a small DistilBERT-based router can learn when retrieval is useful, outperforming both no-retrieve and always-retrieve baselines while using fewer retrievals.
- Scaling to 3000 examples reveals limitations of the SQuAD2 QA backbone and of the heuristic labels, highlighting the need for better QA models and supervision.

Future Work

- Integrate a full DPR index over Wikipedia and compare against the MiniLM retriever used here.
- Fine-tune the QA model on Natural Questions.
- Train the router with richer signals (confidence, answer length, Self-RAG-style critiques)

Code to reproduce results:
https://github.com/anderpnm/router-guided-rag.git